FlowSync AI: SUMO simulation, hand-off brief

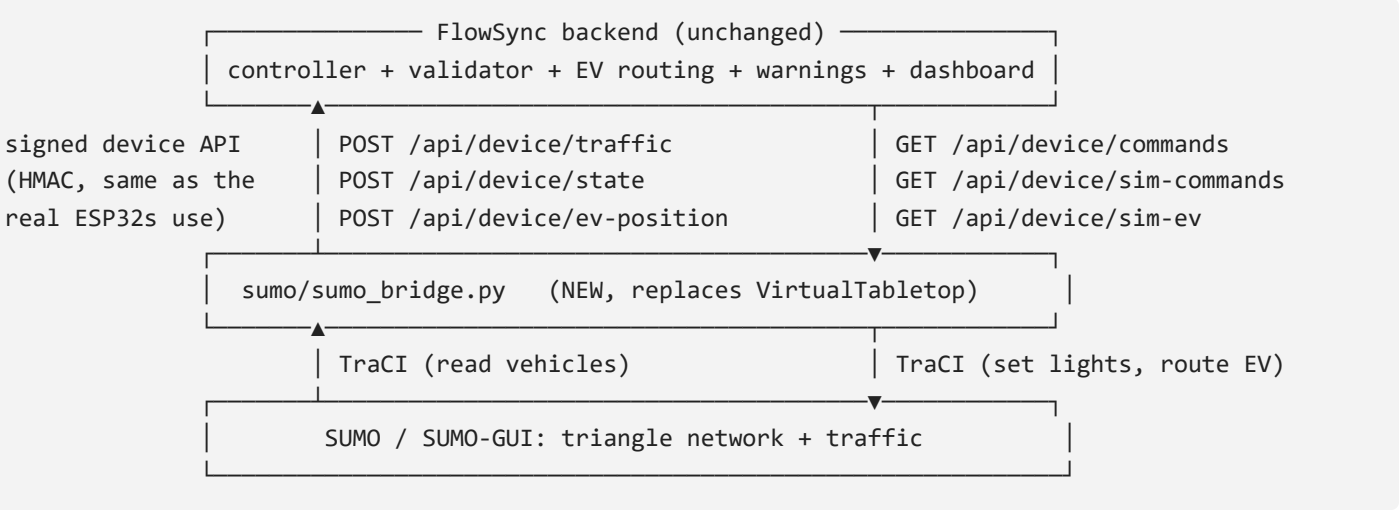
Audience: the teammate (and their coding agent, e.g. Google Antigravity) who will build the SUMO view. **Estimated effort:** 6–8 h for the core (Stages 1–5), plus 2 h for the optional extras. **Golden rule:** SUMO is an *extra view*. It must never be required for the demo. The existing Python virtual tabletop stays as the fallback and must keep working.

---

1. What SUMO will do, end to end

FlowSync controls 3 signalised junctions (I1, I2, I3) joined in a triangle ring, with 3 external entries (West, East, South). The backend already contains the "brain": adaptive signal controller, safety validator, ambulance (EV) routing, EV warnings, incidents, dashboard.

Today the backend is fed by `simulator/tabletop.py` (`VirtualTabletop`): a simple queue-counting toy. **SUMO replaces that toy with a realistic traffic world.** The backend does not change.



Each simulated second, the loop is:

1. **Sense.** Count halting vehicles on each approach edge and send them to the backend as "camera readings".
2. **Decide.** The backend controller picks a target phase per junction. It returns it as a command.
3. **Actuate.** The bridge feeds the command to the existing `SignalFSM`, which enforces min-green, yellow and all-red. The bridge then converts the lamp state into a SUMO traffic-light state string.
4. **Ambulance.** The EV is a SUMO vehicle of class `emergency`. The bridge reports its position to the backend. The backend answers with the best route (live congestion-aware), and the bridge applies it to the EV. Signals ahead of the EV turn green through the backend's normal priority logic.
5. **Scenarios.** The dashboard Scenario panel can still trigger a jam, a stalled vehicle or a camera outage. The bridge translates these into SUMO actions.

What judges see: one SUMO-GUI window with the triangle, normal traffic, an ambulance entering from the West, and signals clearing its path, plus a fixed-timer vs FlowSync comparison with numbers.

---

2. Tooling

- **Eclipse SUMO ≥ 1.20.** Easiest on Windows: `pip install eclipse-sumo traci sumolib` (no MSI needed). Fallback: the official MSI, then set `SUMO_HOME`.
- Python 3.13 (the project's version).
- Put the new dependencies in a separate `requirements-sumo.txt`. Do **not** add them to `requirements.txt`.
- Any coding agent works for this (Antigravity, Claude Code, Cursor, etc.). This document is agent-agnostic. Give the agent this file plus access to the repo.

---

3. Files to create (all under `sumo/`)

```
sumo/
  build_network.py     # topology.yaml -> triangle.nod.xml / .edg.xml / .con.xml -> triangle.net.xml
  routes.py            # generates demand (flows) -> triangle.rou.xml
  sumo_bridge.py       # SumoWorld class: same role as VirtualTabletop
  run_sumo.py          # CLI: python -m sumo.run_sumo [--gui] [--scenario ambulance_demo]
  compare.py           # fixed-timer vs FlowSync metrics (Stage 6)
  gui-settings.xml     # colours, show vehicle ids (optional polish)
  README.md            # how to run
requirements-sumo.txt
tests/test_sumo_*.py   # skipped automatically if SUMO is not installed
```

---

4. Implementation stages

Work in order. Each stage has an **acceptance check**, and a stage is not done until it passes.

Stage 1: Build the network from the topology

Source of truth: config/topology.yaml.

- **Nodes.**
- Junctions I1, I2, I3 are `type="traffic_light"`.
- ext_W, ext_E, ext_S are `type="priority"`.
- Coordinates come from the `layout:` section (e.g. I1 = (240, 170)). Treat 1 layout unit as 1 metre. Flip the y axis if the picture looks mirrored compared with the dashboard.
- **Edges.** Use the exact ids in `edges:` (W1, E2, S3, O1, O2, O3, L12, L21, L13, L31, L23, L32), one lane each, `speed` about 8.9 m/s (a plain urban speed, not the table's 10 cm/s). The geometry (lengths) comes from the node coordinates.
- **Edge id equals approach id.** This is critical. In `approaches:` the inbound edge id is the approach id (W1, L21, L31, E2, L12, L32, S3, L13, L23). The backend identifies every queue by it.
- **Connections.** Give every incoming edge connections to the outgoing edges listed in the road graph. U-turns are not allowed.
- Generate with `netconvert --node-files ... --edge-files ... --connection-files ... -o triangle.net.xml`.

- Do not hand-write the traffic-light logic. Instead, at runtime, discover the controlled links with `traci.trafficlight.getControlledLinks(tl_id)`. Group link indices by their incoming edge's `movement` (A or B) using the `approaches:` table, and build the state string from that grouping. This avoids relying on netconvert's link ordering.

Acceptance: `sumo-gui -n triangle.net.xml` opens. Every id above exists. A script asserts that the set of edge ids equals the set in `topology.yaml`, and that each approach maps to exactly one A or B group.

Stage 2: Traffic demand

- `routes.py` writes `triangle.rou.xml` with `<flow>` entries entering from W1, E2, S3 and leaving via O1, O2, O3 and internal links, with all the routes the topology allows.
- Parameterise by "vehicles per hour per entry" and a seed, so a run is repeatable. Provide three presets: `calm`, `normal`, `rush` (heavy on W1 and L12).
- Define two vehicle types: `car` and `ambulance` (`vClass="emergency"`, `guiShape="emergency"`, red colour, `speedFactor` about 1.2, `lcStrategic` high so it overtakes).

Acceptance: with SUMO's built-in static lights, a 10-minute run completes with no teleports or collisions, and cars reach their exits.

Stage 3: The bridge, read side and signal actuation

Write `SumoWorld` in `sumo_bridge.py` with the same shape as `VirtualTabletop` in simulator/tabletop.py. Study that file first and reuse its `DeviceClient` usage, the signing, and the `SignalFSM` from `backend/app/core/fsm.py`.

Per step (1 simulated second, `traci.simulationStep()`):

1. **Camera readings.** For each approach edge, `count` = number of vehicles on that edge with speed < 0.1 m/s (halting) and `stationary_s` = the longest time any of them has been halting. POST them to `/api/device/traffic`. Include the ambulance in the count, as the real camera does.
2. **State.** POST `/api/device/state` with `{i: fsm.state(now_ms)}` for each junction.
3. **Commands.** GET `/api/device/commands`. For each command, call `fsm.receive_command(...)` and POST the ack, exactly as `tabletop._communicate` does.
4. **Lamps.** Call `fsm.tick(now_ms)`, read `fsm.lamps()` (for example `{"A": "G", "B": "R"}`), then set the SUMO state string with `traci.trafficlight.setRedYellowGreenState(tl_id, state)`. Yellow means `y`, green `G`, red `r`.
5. Never produce two conflicting greens. Assert it in code and fail loudly.

Acceptance: run with the backend up (`uvicorn`, see README). In SUMO-GUI, the lights cycle. The backend dashboard shows queue numbers that match what the GUI shows. Lights do not follow a fixed timer. They favour the busy axis.

Stage 4: Ambulance end to end

- The dashboard (or `--scenario ambulance_demo`) declares an EV, vehicle id `EV-1`, origin ext_W, destination ext_S.
- The bridge inserts a SUMO vehicle `EV-1` of type `ambulance` on the origin edge.

- Each step, report its position: POST `/api/device/ev-position` with `{vehicle_id, edge, progress, source: "simulated"}`. `progress` = lane position divided by lane length, capped at 0.97 (see `tabletop._drive_ev`). Ignore SUMO internal junction edges (ids starting with `:`) by keeping the last real edge.
- Each second, GET `/api/device/sim-ev?vehicle_id=EV-1`. If the returned `route` differs from the EV's current route, apply it with `traci.vehicle.setRoute`, trimmed to start at the current edge. This is what makes the visible rerouting around a jam.
- Priority signalling needs no extra code. The backend's EV logic already asks for the green corridor through the normal command path.
- When the EV leaves the network, remove it and let the backend session end.

Acceptance: run `ambulance_demo`. The ambulance drives W1 → … → S3. When a jam is injected on the direct link, its route visibly changes. It passes every junction without stopping in the best case.

Stage 5: Scenario commands

Poll `GET /api/device/sim-commands` (first call without `after` only returns the cursor, as in `tabletop._control`). Implement these ops, taken from `tabletop.apply`:

op	SUMO action
`reset`	clear all pins, stalls and slowdowns; restore flows
`rates`	scale flow rates on an approach (`traci.vehicle` insertion or restart flows)
`pin`	hold N vehicles on an approach (e.g. a short `setSpeed(0)` hold on N vehicles)
`stall`	stop one vehicle on the edge (`traci.vehicle.setSpeed(vid, 0)`)
`camera`	stop sending `/api/device/traffic` while `on` is false
`link`	stop polling commands, so the local fallback cycle takes over
`drive_ev`	start or stop the EV as in Stage 4

Priority: `camera`, `stall`, `pin`, `reset`. The rest are nice to have.

Acceptance: the Scenario panel's "ambulance through congestion" and "camera loss" scenarios behave sensibly in SUMO.

Stage 6: Comparison metrics (the part that wins judges)

`compare.py` runs the same demand and seed twice:

- **Fixed:** SUMO static lights, or the FSM's `fallback_green_ms` cycle.
- **FlowSync:** the full bridge against a backend instance.

Record these and write a table:

Metric	How
Mean waiting time per vehicle	`traci.vehicle.getWaitingTime`, average at arrival
Mean travel time	per vehicle, exit minus entry
Throughput	vehicles arrived per hour
Ambulance travel time	`EV-1` entry to exit, with and without FlowSync
Ambulance stops	count of times speed < 0.1 m/s

Run at least 5 seeds and report mean ± spread, not a single run. Follow the style of docs/RESULTS.md and of `simulator/evaluate.py`.

Acceptance: `python -m sumo.compare --seeds 5` writes `docs/RESULTS_SUMO.md`. The numbers are reported honestly, even if FlowSync is not better in some case.

Stage 7: Optional extras (only if everything above is green)

- Show the SUMO view in the dashboard (a screenshot or recording loop is enough; do not stream the GUI).
- A `--headless` mode for tests and CI, which uses `sumo` instead of `sumo-gui`.
- `gui-settings.xml`: colour vehicles by speed, label the junctions with the dashboard names.

---

5. Rules (do not break these)

1. **Do not modify the backend** unless you hit a real bug. If you do, tell the project owner and add a test.
2. **Safety is not negotiable.** Every light change goes through `SignalFSM` (min green, yellow, all-red). Never call `setRedYellowGreenState` with a state the FSM did not produce. Never allow conflicting greens.
3. **The SUMO state is SIMULATED.** Label it that way everywhere: `source: "simulated"`, and add a row to docs/BUILT_VS_SIMULATED.md. No claim that the numbers apply to a real road.
4. **Fallback first.** If `import traci` fails or SUMO is missing, the rest of the project must still run and its tests still pass. SUMO tests skip, not fail.
5. **No secrets in the repo.** Device keys come from `config/devices.yaml` through `Settings`, as `run_sim.py` does.
6. Keep `topology.yaml` as the single source of truth. Never copy its numbers into the SUMO code.

---

6. How to run (target end state)

```
# terminal 1: backend
uvicorn app.main:app --app-dir backend --port 8000
# terminal 2: SUMO view with the ambulance demo
python -m sumo.run_sumo --gui --scenario ambulance_demo
# optional: comparison
python -m sumo.compare --seeds 5
```

(Confirm the exact backend start command against the root README. It may differ.)

7. Definition of done

- [ ] `sumo-gui` shows the triangle with the same junction names as the dashboard.
- [ ] Signals respond to the backend and match its queue readings.
- [ ] Ambulance demo works: reroutes around a jam and gets green priority.
- [ ] Camera-loss and stalled-vehicle scenarios work.
- [ ] `docs/RESULTS_SUMO.md` contains the multi-seed comparison.
- [ ] All existing tests still pass, and new SUMO tests pass or skip cleanly.
- [ ] BUILT_VS_SIMULATED.md updated.
- [ ] A 60-second screen recording of the ambulance demo, as a fallback for the live demo.

8. Questions or blockers

If something here contradicts the code, the code wins: report the contradiction to the project owner before changing anything.